Ticket to Ride
Design Document

Authors:
Chay Hong Hong
Josh Brian Saputra
Simao Huang

Team: DevOops

Contents

1 Introduction **3**
2 Team Introduction **3**
2.1 Team Name and Team Photo 3
2.2 Team Membership 4
2.3 Team Schedule 5
2.4 Technology Stack and Justification 5
2.5 Link to the Gitlab repository 6
3 Domain Modelling **6**
3.1 Domain Model 6
3.2 Concept Descriptions 6
3.3 Assumptions and Domain Constraints 8
4 Dynamic Model **9**
4.1 Scenario 1: Game Setup 9
4.2 Scenario 2: End Game and Final Scoring 10
5 Supplementary Documentation **11**

1 Introduction

This document describes the software design for the Ticket to Ride project developed by DevOops. The purpose of this document is to present the structure of the system, the main domain concepts, and how the system behaves during gameplay. The design is documented clearly so that another development team could understand the system and implement it based on this design.

The intended audience for this document includes the project development team, teaching staff assessing the project, and any future developers who may work on the system. The document focuses on explaining the key components of the system and how they interact to support the gameplay mechanics of Ticket to Ride.

The Ticket to Ride system models the core elements of the board game, where players collect train cards, claim routes between cities, and complete destination tickets to earn points. The system manages the game setup, player turns, card drawing, route claiming, and scoring while ensuring that the rules of the game are followed.

This document is organised into several sections. Section 2 introduces the team DevOops, including team members, roles, schedules and the chosen technology stack. Section 3 presents the domain model, which describes the key concepts in the system and their relationships using a UML conceptual class diagram. Section 4 describes the dynamic model, which explains how the system responds to gameplay events using sequence diagrams. Section 5 includes supplementary documentation that helps support and clarify the design.

This document should be read together with the project specification, the team's GitLab repository, and the UML diagrams created during the design process.

2 Team Introduction

2.1 Team Name and Team Photo

Our team name is DevOops. The name reflects the iterative nature of software development, where mistakes ("oops") are part of the learning process and contribute to improving system design and implementation. The team aims to approach development in a structured and professional manner while continuously refining our work.

Moodle Team Name Mapping: CL_Tuesday11AM_31

Team photos



2.2 Team Membership

Our team consists of members with a mix of technical and professional strengths that support the development of the Ticket to Ride project. For Sprint 1, the team lead is the member responsible for organising meetings, coordinating tasks, and ensuring progress is tackled across the team. This role may rotate in later sprints depending on the workload and project needs.

Team Member	Contact Details	Sprint 1 Role / Team Lead	Technical Strengths	Professional Strengths	Fun Fact
Chay Hong Hong	**chon0015@student.monash.edu**	**Team Lead**	**Python and Java, Ui design, problem solving**	**Communication, Organisation, Teamwork.**	**Can use both hands, can't speak chinese.**
Josh Brian Saputra	**jsap0007@student.monash.edu**	**Developer**	**Coding, problem solving, design**	**Adaptability, teamwork**	**Looks chinese cant speak chinese**
SimaoHuang	**shua0115@student.monash.edu**	**Developer**	**UML diagrams, system design, coding**	**Teamwork**	**Speak chinese**

2.3 Team Schedule

Our team has established a regular meeting and work schedule to support steady progress throughout the sprint. Weekly meetings are used to review completed work, discuss blockers, and allocate upcoming tasks. In addition to scheduled meetings, members also communicate through online messaging like discord to handle smaller updates between meetings.

Our team meeting is held each week around our Tuesday 11AM workshop schedule, with additional discussions taking place online when necessary. Members are expected to complete their allocated task before the next team check-in so that progress can be reviewed consistently.

Workload will be distributed based on each member's technical strengths, confidence with assigned task, and availability during the sprint. For Sprint 1, responsibilities are shared across team introduction content, technology prototyping, domain modelling, and interaction diagrams. Progress will be managed through regular check-ins and the Git repository all members can track contributions and see if there are any unfinished tasks.

2.4 Technology Stack and Justification

Our team plans to use Python as the primary programming language for this project. Python is a suitable choice because it supports object- oriented programming and allows the team to implement the key entities and behaviours of Ticket to Ride, such as players, routes, cities, cards and overall game flow. It is also an appropriate choice for developing a standalone game application for this assignment.

Python matches our team's current experience, as team members already have some familiarity with it from previous coding tasks and university work. This makes it a practical choice because the team can spend more time focusing on the game design and implementation rather than learning a new programming language from scratch. Python can also be used with Tkinter, which is one of the approved GUI options for early sprints.

Java was considered as an alternative programming language due to its strong object-oriented capabilities and suitability for larger software systems. However, the team decided not to use Java because it has a higher setup and development overhead compared to Python. Additionally, Python is more familiar to all team members, allowing faster development and reducing the time required to learn language-specific syntax and tooling. For the scope of this project, Java was therefore not selected in favour of a more efficient and accessible development process.

Overall, our team selected Python because it is the most suitable choice based on our current skills and the requirements of the assignment. It is simple to work with, supports the development style we need, and will allow us to build toward a working standalone game in the future sprints.

Each team member will also complete a small Python prototype and upload it to the GitLab repo in the prototype folder. This will help confirm that all members can use Python in practice and can create code that runs successfully before moving into later sprints.

2.5 Link to the Gitlab repository

https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/CL_Tuesday11AM_31

3 Domain Modelling

3.1 Domain Model

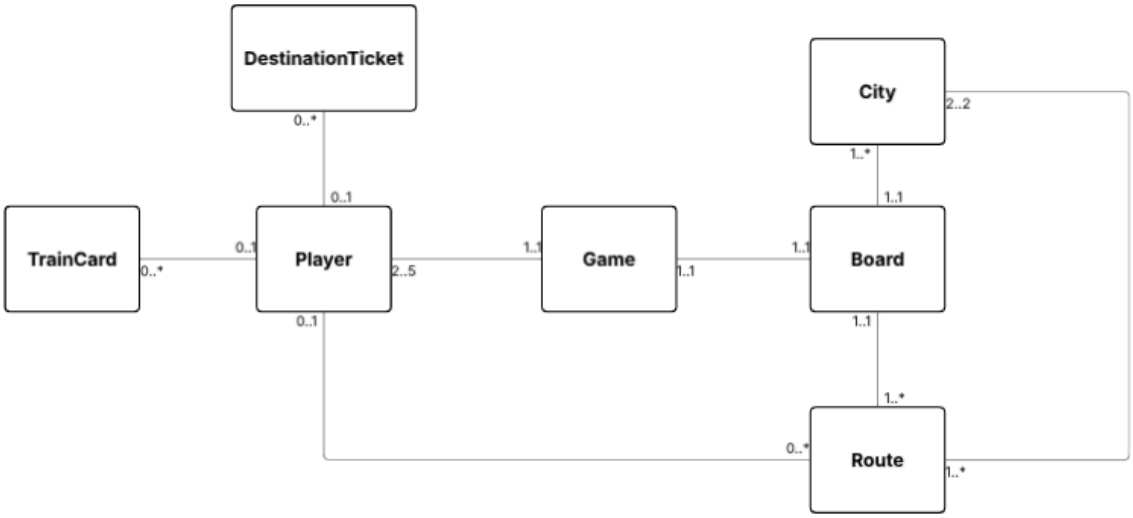


Figure 1: Conceptual Model of the System

3.2 Concept Descriptions

Concept: Game
Description
Represents a single session of the game where multiple players compete to build railway routes, collect cards, and complete destination tickets to earn points.

Relationships

- Contains 2..5 Players, representing the participants in the game.
- Uses 1 Board, which defines the map of cities and routes used during the game.

Concept: Player
Description
Represents an individual participant in the game. Players collect train cards, claim routes on the board, and attempt to complete destination tickets to earn points.

Relationships
- Participates in 1 Game.
- Claims 0..* Routes on the board.
- Holds 0..* TrainCards used to claim routes.
- Holds 0..* DestinationTickets representing objectives.

Concept: Board
Description
Represents the map used during the game. The board contains cities and routes connecting them, forming the network that players attempt to build across.

Relationships
- Belongs to 1 Game.
- Contains 1..* Cities.
- Contains 1..* Routes.

Concept: City
Description
Represents a location on the board that can be connected to other cities through routes. Cities serve as endpoints for routes and destinations for players attempting to complete tickets.

Relationships
- Belongs to 1 Board.
- Connected to 1..* Routes.

Concept: Route
Description
Represents a railway connection between two cities on the board. Players may claim routes by using the required train cards, allowing them to build part of their railway network.

Relationships
- Connects exactly 2 Cities.
- Belongs to 1 Board.
- May be claimed by 0..1 Player.

Concept: TrainCard
Description
Represents a colored card used by players to claim routes on the board. Different routes require different numbers or types of train cards.

Relationships
- Held by 0..1 Player.

Concept: DestinationTicket
Description
Represents a goal card that requires a player to connect two specific cities through their claimed routes. Completing the connection awards points, while failing to complete it results in a penalty.

Relationships
- Held by 0..1 Player.

3.3 Assumptions and Domain Constraints

The following assumptions and domain constraints were used when defining the conceptual domain model.

- A game must contain between 2 and 5 players, consistent with the rules of the game.
- Each game uses exactly one board, which defines the cities and routes available.
- Each board contains multiple cities and routes forming the playable network.
- A route always connects exactly two cities.
- Each city must connect to at least one route to ensure it is reachable within the network.
- A route can only be claimed by one player during a game.
- A player may claim multiple routes throughout the game.
- Players may hold multiple train cards and destination tickets simultaneously.

4 Dynamic Model

4.1 Scenario 1: Game Setup

Scenario Description

This scenario describes how a new game session is initialized. Players join the game, the board is prepared, and initial resources are distributed. The system then transitions into the main gameplay state where players can begin taking turns.

Sequence Diagram

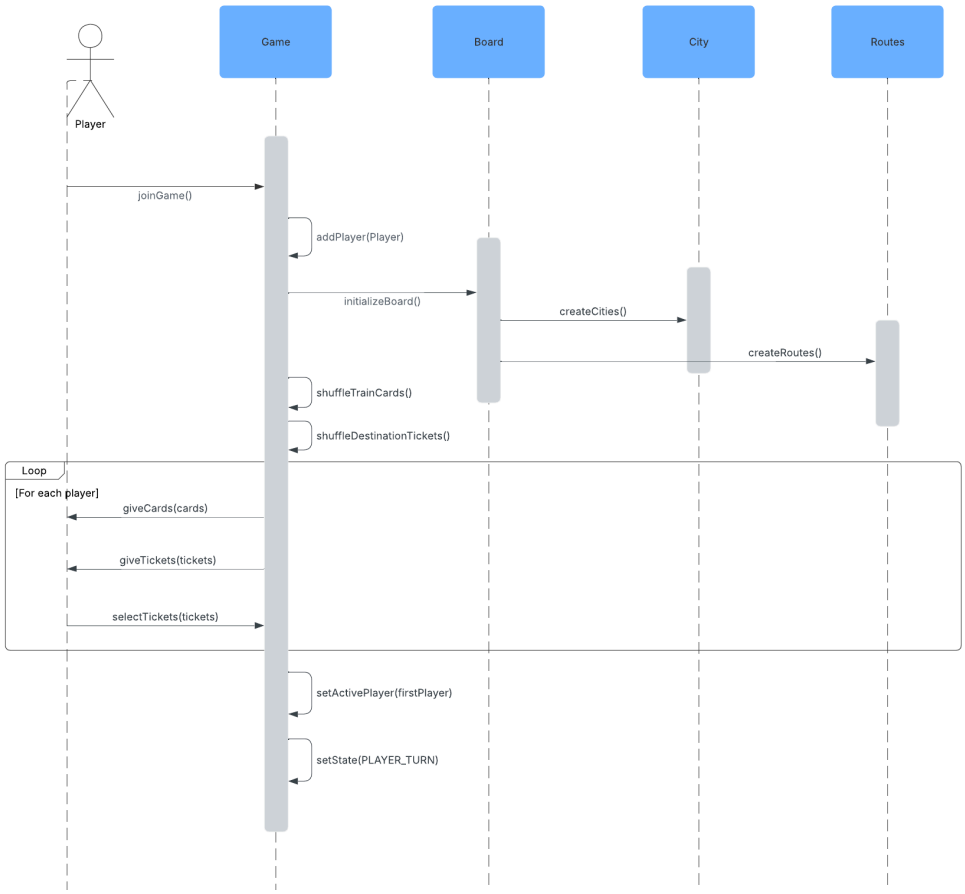


Figure 2.1: Sequence Diagram of Game Setup

4.2 Scenario 2: End Game and Final Scoring

Scenario Description

This scenario begins when the end-game condition is triggered (e.g., a player has a low number of remaining trains). The system allows all players to complete their final turn before calculating final scores, evaluating destination tickets, awarding bonuses, and determining the winner.

Sequence Diagram

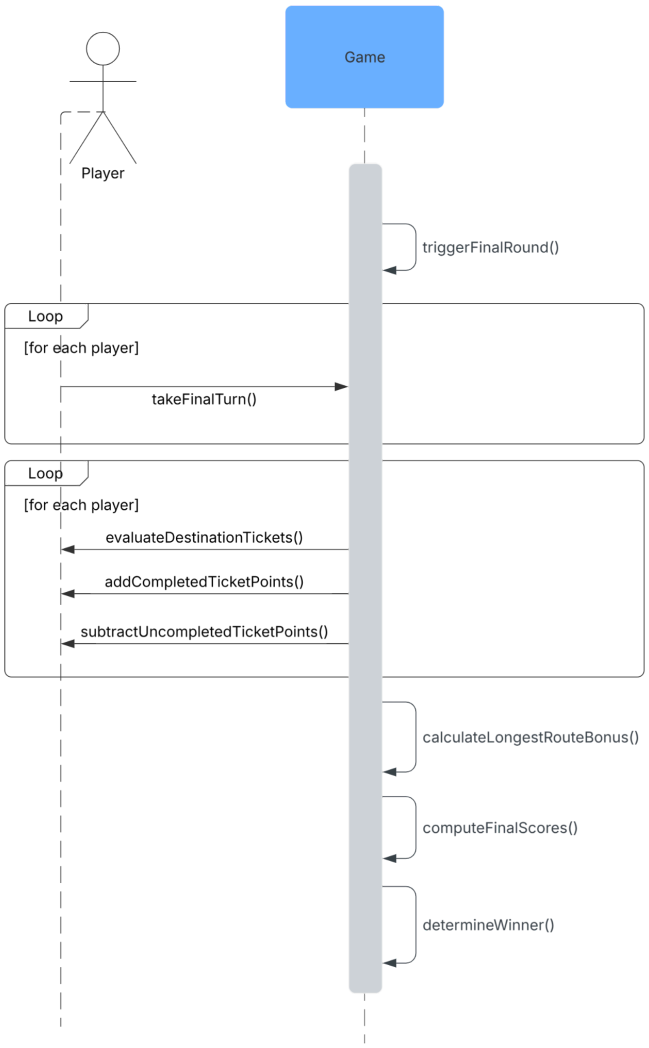


Figure 2.2: Sequence Diagram of Game End

5 Supplementary Documentation

- DevOops Gitlab Repository: https://git.infotech.monash.edu/FIT3077/fit3077-s1-2026/assignment-groups/CL_Tuesday11AM_31
- Lucidchart UML Diagrams: https://lucid.app/lucidchart/3fd1e90a-dbd4-40c7-8475-ec1422a75a35/edit?viewport_loc=-1382%2C-1042%2C2558%2C1232%2C0_0&invitationId=inv_e711241a-aff0-406c-ba83-b2e99622d518
- Lucidchart Sequence Diagrams: https://lucid.app/lucidchart/3b216d17-79e4-4a80-beb2-b99cc28b15a5/edit?viewport_loc=253%2C971%2C3256%2C1568%2C0_0&invitationId=inv_c388fdcc-6ecb-4c6c-8368-b77b7ac73023
- Official Ticket to Ride Game Rules: https://cdn.1j1ju.com/medias/2c/f9/7f-ticket-to-ride-rulebook.pdf